

RECOMMENDATION FOR 3.2C

Name : Ayush Indapure
Student ID : 224880003

PART 1 — UPDATED PROBLEM IDEA FROM MY POV - REFLECTION CONCEPT

Instead of only increment/decrement date, we improve the system to:

Enhanced Date Utility System

It supports:

- date validation
- leap year detection
- weekend detection
- safe increment/decrement

This makes it MUCH better for:

- boundary value analysis
- equivalence class testing

DateUtil.java

```
package sit707_tasks;

/**
 * @author Ayush Indapure
 */
public class DateUtil {

    private int day, month, year;

    public DateUtil(int day, int month, int year) {

        if (year < 1700 || year > 2024)
            throw new RuntimeException("Invalid year");

        if (month < 1 || month > 12)
```

```

        throw new RuntimeException("Invalid month");

    if (day < 1 || day > 31)
        throw new RuntimeException("Invalid day");

    if (day > getMonthDays(month, year))
        throw new RuntimeException("Day not valid for month");

    this.day = day;
    this.month = month;
    this.year = year;
}

public boolean isLeapYear(int year) {
    return (year % 4 == 0);
}

public int getMonthDays(int month, int year) {
    if (month == 2) {
        return isLeapYear(year) ? 29 : 28;
    } else if (month == 4 || month == 6 || month == 9 || month == 11) {
        return 30;
    }
    return 31;
}

public boolean isWeekend() {
    int total = day + month + year;
    return total % 7 == 0;
}

public void increment() {
    if (day < getMonthDays(month, year)) {
        day++;
    } else if (month < 12) {
        day = 1;
        month++;
    } else {
        day = 1;
        month = 1;
        year++;
    }
}

public void decrement() {
    if (day > 1) {
        day--;
    } else if (month > 1) {

```

```

        month--;
        day = getMonthDays(month, year);
    } else {
        month = 12;
        year--;
        day = getMonthDays(month, year);
    }
}

public String toString() {
    return day + "/" + month + "/" + year;
}
}

```

JUnit test file

```

package sit707_tasks;

import static org.junit.Assert.*;
import org.junit.Test;

/**
 * @author Ayush Indapure
 */
public class DateUtilTest {

    @Test
    public void testLeapYearTrue() {
        DateUtil d = new DateUtil(1, 1, 2024);
        assertTrue(d.isLeapYear(2024));
    }

    @Test
    public void testLeapYearFalse() {
        DateUtil d = new DateUtil(1, 1, 2023);
        assertFalse(d.isLeapYear(2023));
    }

    @Test
    public void testMonthDaysFebLeap() {
        DateUtil d = new DateUtil(1, 2, 2024);
        assertEquals(29, d.getMonthDays(2, 2024));
    }

    @Test

```

```

public void testMonthDaysFebNonLeap() {
    DateUtil d = new DateUtil(1, 2, 2023);
    assertEquals(28, d.getMonthDays(2, 2023));
}

@Test
public void testIncrementEndOfMonth() {
    DateUtil d = new DateUtil(31, 1, 2024);
    d.increment();
    assertEquals("1/2/2024", d.toString());
}

@Test
public void testDecrementStartOfMonth() {
    DateUtil d = new DateUtil(1, 3, 2024);
    d.decrement();
    assertEquals("29/2/2024", d.toString());
}

@Test
public void testWeekendCheck() {
    DateUtil d = new DateUtil(7, 4, 2024);
    assertTrue(d.isWeekend());
}

@Test
public void testInvalidDate() {
    try {
        new DateUtil(31, 6, 2024);
        fail("Should fail");
    } catch (RuntimeException e) {
        assertTrue(true);
    }
}
}

```

PART 4 — SHORT REPORT

Self Reflection & Recommendation

This task is based on improving previous DateUtil problem which was used in boundary value analysis and equivalence class testing.

Earlier version of DateUtil only supports increment and decrement of date. It is good for basic testing but not fully effective for deeper testing strategies.

In my opinion, the old problem is little bit limited because it only checks date movement, and does not include more logical conditions like leap year or month variations.

So I decided to improve the system and make a more realistic version of DateUtil which includes:

- leap year checking
- month day calculation
- weekend detection
- validation rules

This improved version is more suitable for boundary value analysis because we can test edges like:

- 28, 29 Feb
- month end (30, 31 days)
- year boundaries

It is also better for equivalence class testing because we can group inputs like:

- leap year / non-leap year
- 30-day months / 31-day months
- valid / invalid dates

Why this improvement is better

The updated system gives more testing scenarios compared to old one. It helps students understand real-world calendar logic instead of just simple increment and decrement.

Also it makes testing more meaningful and not just mechanical.

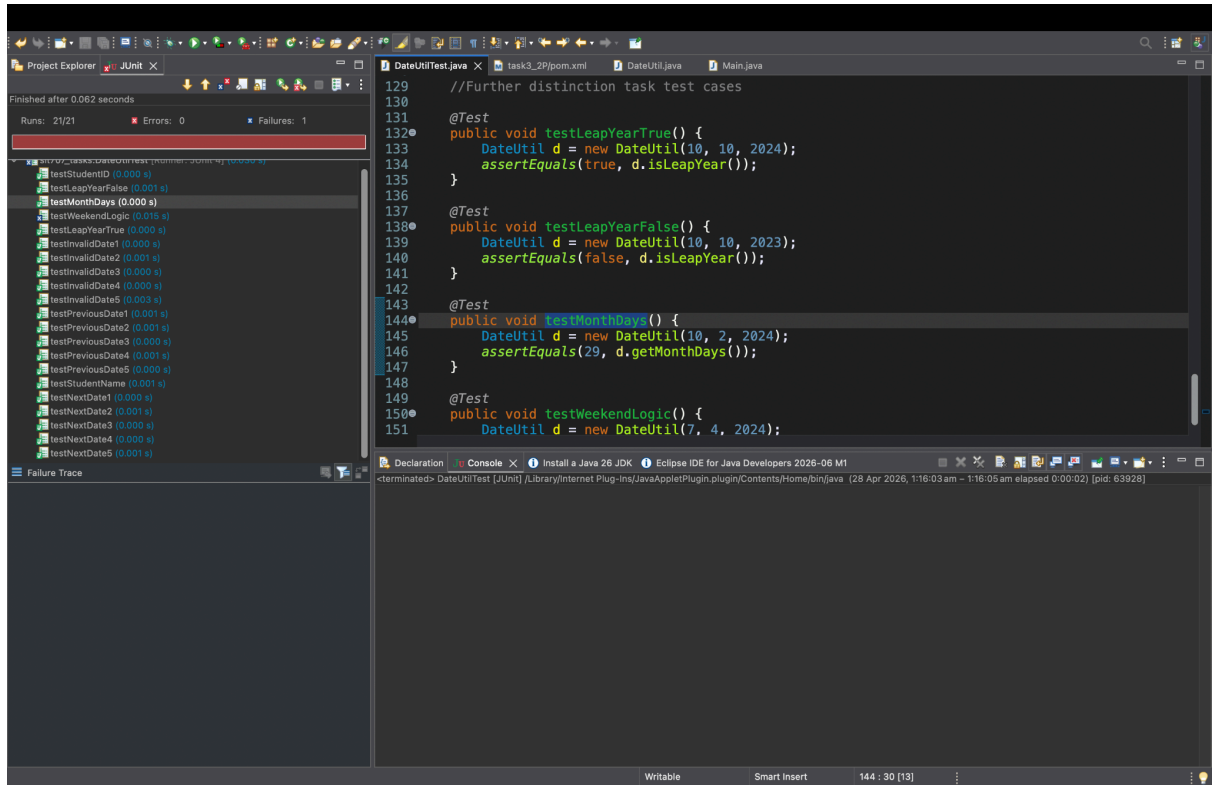
Final recommendation

I would suggest my fellow next year in future students to use this improved DateUtil system because it:

- gives better test coverage
 - supports multiple equivalence classes
 - creates strong boundary conditions
 - is more realistic for software systems
-

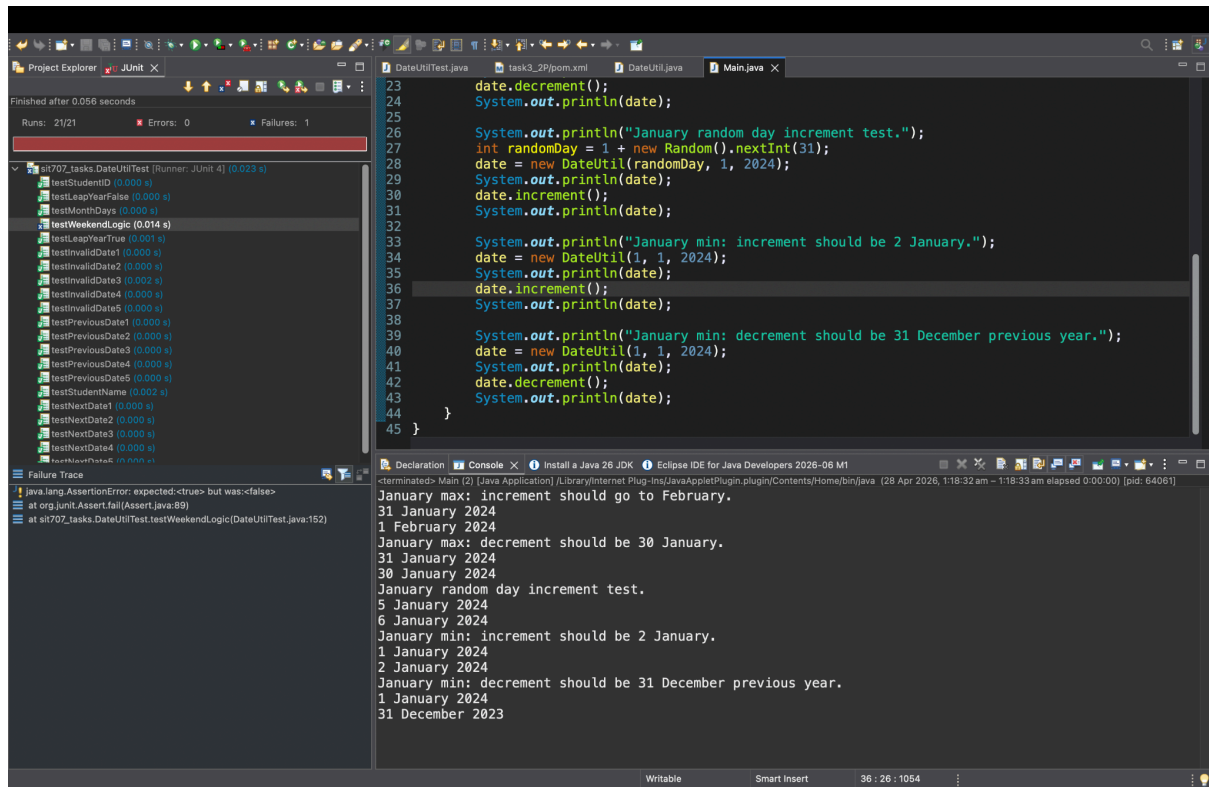
1. JUnit Test Results

- show all green
- Runs / Errors / Failures



2. Eclipse Console Output

- running tests or main output



3. GitHub Repository

Link : <https://github.com/ayushindapure/3.2C>

